

P0860r0 : Atomic Access Property for span and mdspan

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0860r0
Date: 2018-02-10
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Daniel Sunderland
Contact: dsunder@sandia.gov
Audience: Parallelism and Concurrency Study Group (SG1)
Audience: Library Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0860.rst

1 Revision History

1.1 P0856r0

2017-11-Albuquerque LEWG feedback for P0009, P0019, and P0546

- Extract atomic array reference from P0019 to a separate paper

2 Description / Motivation

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An `atomic_ref` (P0019) is used to perform atomic operations on the non-atomic members of the referenced array. Construction of an `atomic_ref` for a non-atomic object requires the non-atomic object satisfy several conditions and potentially must acquire resources such as a lock in order to perform atomic operations on that object. Construction of `atomic_ref` has potentially large overhead which could be performed once for an entire span.

The proposed `mdspan` (**P0009**) allows for additional property parameters to be specified. Collections of property parameters form an ontology, described in **P0900**, where the *property-values* are organized into orthogonal *property-types*.

We propose the `atomic_access` property-value for span and mdspan such that

- all references to `mdspan` elements are `atomic_ref`
- potential overhead associated with constructing `atomic_ref` occurs only once when an `mdspan` object is constructed.

3 Proposal

Modify `mdspan` specification:

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

    template< class ArrayType , typename ... Properties >
    struct mdspan {
        // revision
        // using reference = element_type & ; // remove
        using reference = /* implementation defined */ ;
    };

}}}
```

Add to **[views]** a new **[views.atomics]** section.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

    struct atomic_access ;

}}}
```

The `atomic_access` property-value may appear at most once in the `Properties...` parameter pack of `span` or `mdspan`. Let S be an instantiation of `span` or `mdspan` with `Properties...` parameter pack containing `atomic_access`. Let R be `atomic_ref< S::element_type>`.

Requires:

- $S::\text{element_type}$ satisfies requirements for `atomic_ref< S::element_type>`,
- S constructors also require R constructors' requirements for member objects of S .

Effects:

- $S::\text{reference}$ is `atomic_ref< S::element_type>`,
- S iterator types dereference to `atomic_ref< S::element_type>`

4 Application to span

When the `span` library is updated to accept properties (**P0546**) the same revision should be applied to accept the `atomic_access` property-value.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

    template< class ArrayType , typename ... Properties >
    struct span {
        // revision
```

```
// using reference = element_type & ; // remove  
using reference = /* implementation defined */ ;  
};  
}}
```